Flask + Machine Learning + Git + GitHub + Render: Full Training Manual

Introduction

This manual explains the complete lifecycle of deploying a machine learning model: training a model, saving it as a .pkl file, building a Flask API, testing locally, managing source code with Git, storing code in GitHub, deploying to Render, and testing the deployed endpoint.

Project Architecture

Flow: Training Data -> Linear Regression Model -> house_model.pkl -> Flask API -> Git Repository -> GitHub -> Render Deployment -> Public URL -> Client Requests.

Understanding train_model.py

The purpose of train_model.py is to train the model once and save it. Pandas is used to create a DataFrame. LinearRegression learns the relationship between house size and price. model.fit(x,y) trains the model. joblib.dump() saves the trained model to disk.

Line-by-Line Explanation

import pandas as pd imports Pandas. from sklearn.linear_model import LinearRegression imports the algorithm. x contains input features. y contains target values. model.fit() learns the relationship. joblib.dump() serializes the model.

Understanding app.py

Flask exposes the model as an API. app = Flask(__name__) creates the application. joblib.load() loads the saved model. @app.route('/') creates the home endpoint. @app.route('/predict') creates the prediction endpoint.

Request Flow

A client sends JSON such as {'size':1700}. Flask reads request.json. The size value is extracted and passed to model.predict(). The result is returned through jsonify().

Why Use APIs?

Without an API, users would need direct access to Python code. APIs allow web apps, mobile apps, and other systems to obtain predictions through HTTP requests.

requirements.txt

requirements.txt lists dependencies. Render reads this file and installs packages automatically. It ensures reproducible deployments.

Local Testing

Run python app.py. Visit http://127.0.0.1:5000. Test POST requests using requests or Postman before deploying.

Git Fundamentals

Git tracks source code changes. A repository stores project history. Commits create checkpoints. Branches allow parallel development.

Git Commands Explained

git init creates a repository. git add . stages files. git commit saves changes. git branch -M main renames the branch. git remote add origin links GitHub. git push uploads code.

GitHub Fundamentals

GitHub hosts repositories remotely. It acts as a backup and collaboration platform. Render deploys directly from GitHub repositories.

Creating a GitHub Repository

Create a repository, copy the repository URL, then connect your local repository using git remote add origin.

Render Deployment

Create a Render Web Service. Connect GitHub. Choose the repository. Build command installs dependencies. Start command launches Gunicorn.

Why Gunicorn?

Flask's built-in server is intended for development. Gunicorn is a production WSGI server designed to handle web traffic reliably.

Build Command

pip install -r requirements.txt installs all dependencies during deployment.

Start Command

gunicorn app:app means run the Flask object named app inside app.py.

Testing the Production Endpoint

Use requests.post() with JSON data. Verify status code 200 and inspect the returned JSON.

Common Error: JSONDecodeError

This often occurs when the server returns HTML instead of JSON. Causes include wrong endpoint, server errors, missing keys, or application crashes.

Your Specific Error

Your API expected {'size':1700} but the client sent feature1, feature2, feature3. Flask raised KeyError('size'). Render returned an HTML error page. response.json() attempted to parse HTML and failed.

Improving Robustness

Validate inputs before prediction. Return informative JSON error messages instead of allowing exceptions to propagate.

Updating a Deployment

After modifying code: git add ., git commit -m 'message', git push origin main. Render automatically redeploys.

Production Considerations

Real-world systems may use databases, authentication, monitoring, logging, CI/CD pipelines, containers, and Kubernetes.

Interview Questions

What is Flask? What is an API? Why save models using joblib? Why use Gunicorn? Difference between local deployment and cloud deployment? Explain git add, commit, push. Explain request-response lifecycle.

Conclusion

You now have a complete understanding of training a model, exposing it through Flask, managing it with Git, storing it in GitHub, deploying it on Render, and testing it from clients.